

COMP 10260: Cloud Computing with ReST

SAM SCOTT, MOHAWK COLLEGE, JUNE 2023¹

“**Cloud computing**” is a new name for an idea that dates back to the dawn of computer networking in the 1960’s. The idea is that sometimes a program running on a client machine needs access to computing power, code, or stored data on a server machine, and it’s useful to be able to ask that server machine to run some code, perform some computations, or retrieve and update your data through a **Remote API (Application Programming Interface)**.

Remote APIs have been standardized in many ways over the years, and form the basis of cloud computing. Cloud computing services that define their API using HTTP requests and responses are known as **web services**, and by far the most popular web service standard is **REST (REpresentational State Transfer)**².

Well-known examples of Cloud Computing via RESTful APIs include:

- The Twitter and Discord APIs, which allow developers to create bots that read, update, delete, and post messages on Twitter and Discord;
- The OpenAI API, which allows developers to train and run massive, generative neural networks in response to custom data and prompts; and
- Amazon Web Services, which offers a huge variety of cloud-based computing services over the web via RESTful APIs.

We’ll use the Twitter API as an example throughout: <https://developer.twitter.com/en/docs/twitter-api>

How a RESTful API Works

REST is a loose set of standards for how a client should make remote API calls via **HTTP requests** to a web service in the “cloud” (i.e., on a server connected to the Internet) and receive the results of those API calls via **HTTP responses**.

When making a remote API call using an HTTP request, the client sends information in the following ways:

- a. in the **path** of the URL,
- b. through the choice of **request method**,
- c. in the **body of the request** (often JSON-encoded), and
- d. in **query parameters** (a.k.a. GET Parameters).

¹ Based on original materials by Jonathan Graham.

² The idea behind the name is that the server machine **transfers a representation** of its current **state** (e.g., the contents of database tables) to the client machine. Contemporary uses of web services for cloud computing go well beyond this simple idea but the name has stuck.

When responding with an HTTP response, the server sends Information in the following ways:

- e. using the **HTTP response code** to indicate success or failure, and
- f. in the body of the **HTTP response** (often JSON-encoded).

a. The URL

Suppose you are creating a RESTful API to retrieve student data. It's going to run over the web, so you need a **web server** and a **domain** for it. Let's say you own **webservices.com**. You might install your student information **web service** at **https:// webservices.com/my_api** using PHP as the programming language.

Here's what some of that API might look like. Note that the URLs listed can include parameters as part of the URL itself. These are marked using a colon (e.g., ":id") and are referred to as **path parameters** to distinguish them from the **query parameters** that follow the question mark in a URL.

Method	URL	Description
GET	/my_api/student_id	Retrieve a list of valid student ids.
GET	/my_api/student_id/:id	Retrieve the information for a single student id.

According to the above API, if someone wants to get the list of all valid student ids, they send the following HTTP GET request:

```
https://webservices.com/my_api/student_id
```

This URL will redirect to a PHP script file that will look up all the valid student IDs (presumably in a database). If all goes well, it will send back an **HTTP response** with response status code **200 (OK)** and a JSON-encoded array of student IDs in the body of the response. It might look something like this...

```
200 OK      ["000123456", "1239995555"]
```

If a user wants to retrieve the information for a particular student, they send the following GET request, substituting a valid student id for the **:id** path parameter:

```
https://webservices.com/my_api/student_ids/000123456
```

And the PHP script receiving the code might respond as follows, with a JSON-encoded object in the body of the request:

```
200 OK      {"id":"000123456", "name":"Sam Scott", "average":85}
```

Twitter Example

For an example of how Twitter uses GET requests in its API, go to

<https://developer.twitter.com/en/docs/twitter-api/tweets/lookup/api-reference>.

Implementing the Uniform Interface

Note that the URLs in the API are being used in a very different way. Normally if you send a request for **/my_api/student_id/000123456** you are asking for the contents of a folder called **000123456**. But in this case, **000123456** is a piece of data, a parameter for the request, and not the name of a resource.

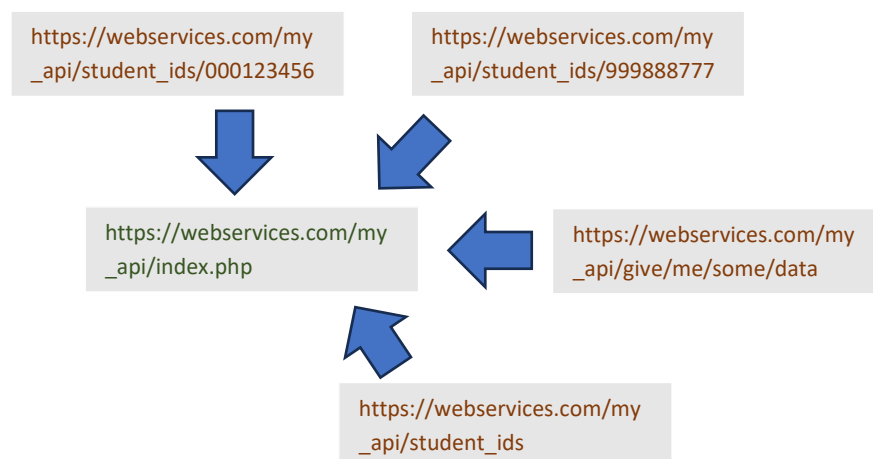
The best way to make this setup work is to have all URLs involving the **my_api** folder re-routed to single PHP control script. Then that script reads the original URL and uses PHP string processing functions to figure out what the request is and retrieve any **path parameters** from the URL.

You can redirect URLs on an Apache web server by putting a file named **.htaccess** inside the **my_api** folder. The contents of the **.htaccess** file should activate Apache's **rewrite engine** to point every request for that folder to a single file.

Here's an example of the **.htaccess** file you would place inside the **my_api** folder:

```
RewriteEngine On
RewriteRule . index.php
```

The **“.”** is a wild card, so this rule says to redirect any URL that enters the **my_api** folder to the file **my_api/index.php** as shown below...



Hands-On Activity

You can see a uniform interface in action using the **rest_api_basics** example on Canvas. Place the **rest_api_basics** folder into your **htdocs** folder, then try it out through the browser. No matter what you type in the URL after **http://localhost/rest_api_basics**, it will still run **index.php**.

PHP Support for the Uniform Interface

The control script will need to know the path from the original request. In PHP, this information is stored in the `$_SERVER` object:

```
$path = $_SERVER[ 'REQUEST_URI' ];
```

To separate the different parts of the path and retrieve path parameters, you need to use string processing functions. For example, these two lines will separate the “request” path from the base URL:

```
$prefix = "/rest_api_basics";  
$request = substr($path, strlen("$prefix"));
```

The above code (from `rest_api_basics`) uses two different PHP string processing functions:

Function	Explanation
<code>strlen(\$s)</code>	Returns the length of <code>\$s</code>
<code>substr(\$s, \$offset, [\$length])</code>	Returns the substring of <code>\$s</code> starting at <code>\$offset</code> . Use an optional third parameter to limit the <code>\$length</code> of the substring.

For a full list of PHP string functions go to <https://www.php.net/manual/en/ref.strings.php>.

In the response from PHP, you can echo a JSON-encoding of whatever data you wish to send. You can use the same `json_encode` function that you learned about in AJAX for that:

```
echo json_encode($data);
```

b. The Request Method

RESTful APIs make use of HTTP “**verbs**” (request methods) to indicate the operation they are being asked to perform. We’re already familiar with HTTP request methods **GET** and **POST**, but there are several other methods that are also available to be used.

In this course we’ll use the following four methods, corresponding to the four basic SQL row operations:

Method	Description	SQL Operation
GET	Retrieve information	SELECT
POST	Add new information	INSERT
PUT	Update existing information	UPDATE
DELETE	Delete existing information	DELETE

Twitter Examples

For examples of how Twitter uses POST and DELETE requests in its API, go to

<https://developer.twitter.com/en/docs/twitter-api/tweets/manage-tweets/api-reference>.

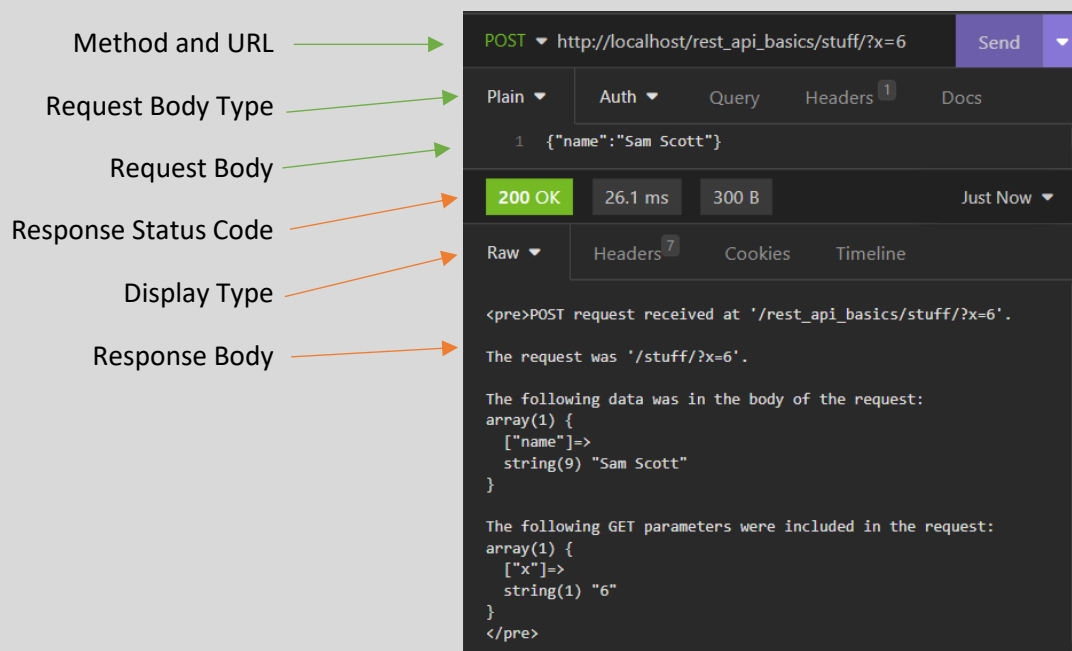
For examples of how Twitter uses PUT requests in its API, go to

<https://developer.twitter.com/en/docs/twitter-api/tweets/hidden-replies/api-reference/put-tweets-id-hidden>.

Hands-On Activity

Try using an API development tool such as **Insomnia** or **Postman** to send PUT, POST, and DELETE requests to **http://localhost/rest_api_basics_full** and read the result.

To download **Insomnia**, go to <https://insomnia.rest/download>. Then run Insomnia and create a **new collection**, then click a **new HTTP request**. You can type the URL (including query parameters). You can configure and send requests as shown below.



PHP Support

Like the path information, the request method is also stored in the `$_SERVER` object and can be retrieved as follows:

```
$method = $_SERVER[ 'REQUEST_METHOD' ];
```

You should be able to find the above code in `rest_api_basics_full`.

A Full Student Data REST API

Here's an expanded version of the student data API. In this version, different request methods are used to specify what action to take for each URL.

Method	URL	Description
GET	/my_api/student_id	Retrieve a list of valid student ids.
GET	/my_api/student_id/:id	Retrieve the information for a single student id.
POST	/my_api/student_id/:id	Add a new student record using :id.
PUT	/my_api/student_id/:id	Modify a student record by :id
DELETE	/my_api/student_id/:id	Delete a student record by :id

Now the same URL has a different meaning depending on the request method. The URL below will either retrieve, insert, update, or delete student 000123456 depending on the request method used:

```
https://webservices.com/my_api/student_id/000123456
```

Hands-On Activity

The `example_rest_api` code folder on Canvas contains a toy implementation of the above API. If you unzip it to your `htdocs` folder, you can use **PostMan** or **Insomnia** to send requests like this:

```
http://localhost/example_rest_api/student_ids
http://localhost/example_rest_api/student_ids/000123456
```

Try all four request methods and compare the responses. Note that this API doesn't actually do anything – there is no database being queried or updated. It's just simulating the operations and returning text information as a result.

PHP Support

The code in **example_rest_api** uses `$_SERVER` to get the path and the method and uses the **strlen** and **substr** functions to work with the path string. It also uses **strpos** to determine whether a URL with path parameters is well formed:

Function	Explanation
strpos(\$s, \$t)	Returns the index of \$t inside \$s, or false if \$t is not found.

The code below makes sure that the path is correct before using **strlen** and **substr** to retrieve path parameters:

```
$found = strpos($path, "$prefix/student_id/");  
if ($found === 0) { ... }
```

c. The Request Body

POST and PUT operations require data to be sent along with the request. This data is typically sent in the body of the HTTP request. It can be formatted in any way you like, but JSON is always a good choice!

Hands-on Activity

Send PUT and POST requests to **example_rest_api** using **Insomnia** or **PostMan**. Change the **Body** type to **JSON** and hand-code some JSON for your POST or PUT request. For example:

```
{ "name": "Sam Scott", "average": 99 }
```

PHP Support

The code in **example_rest_api** uses **file_get_contents("php://input")** and **json_decode** to read and decode the body of the request:

```
$data = json_decode(file_get_contents('php://input'), true);
```

The optional second parameter for **json_decode** specifies that objects in the JSON should be decoded to **associative arrays**, rather than to PHP objects.

It is also a good idea to set headers in the HTTP response. The PHP **header** function can be used for this purpose. If you are sending JSON data, consider setting this header:

```
header("Content-Type: application/json");
```

d. Query Parameters

Another way to get information into an API request is with **query parameters** in the URL. This will work no matter what the request method. Typically, this is used for searching or similar operations that filter rather than identify a resource.

For example, you might want the user to be able to search by student name. Then you might define the following:

Method	URL	Description
GET	/my_api/student_ids/search/?q=:name	Retrieve the id for a student with a matching :name.

Twitter Example

For an example of how Twitter uses query parameters in its API go to

<https://developer.twitter.com/en/docs/twitter-api/tweets/search/api-reference/get-tweets-search-recent> and scroll down to the **query parameters** section.

Hands-On Activity

Send query parameters to **rest_api_basics_full** using Insomnia. Try different HTTP request methods.

PHP Support

The code in **rest_api_basics_full** accesses the query parameters through the **\$_GET** method, as we have been doing for the entire course.

e. HTTP Response Status Codes

RESTful APIs use HTTP response status codes to communicate an overall status for success or failure of an API call. They use 200-level codes for successful operations, 400-level codes for unsuccessful operations, and 500-level codes for catastrophic failures (e.g. unable to connect to the database).

Here's a list of the most common and useful response codes:

Code	Name	Standard Interpretation
200	OK	Everything went great! (for GET)
201	CREATED	The resource has successfully been created (for POST)
204	NO CONTENT	Everything went great but I have no data for you (for DELETE and PUT)
400	BAD REQUEST	Something went wrong with the request. Perhaps the URL was malformed or a path or query parameter was missing or incorrectly formatted.
401	UNAUTHORIZED	You're not allowed to do that!
404	NOT FOUND	The resource you are trying to GET or PUT to doesn't exist.
405	METHOD NOT ALLOWED	The method used to send the request is invalid.
500	INTERNAL SERVER ERROR	There's a problem on the server (exception, database connectivity error, etc.).

Hands-On Activity

Using Insomnia, try to send queries to **example_rest_api** that will trigger a 201, 400, and 404 response status code.

PHP Support

The code in **example_rest_api** uses the **http_response_code** function to set a response code when necessary.

```
http_response_code(400);
```

If no call is made to **http_response_code**, the response code will be **200 (OK)**.

Client-side Code Support for ReST APIs

JavaScript Support

You can use the **fetch** command to make an AJAX call to a RESTful API. The code below is from **test_rest_api_basics/test.html** on Canvas.

```
function success(data) {
    let div = document.getElementById("myDiv");
    div.innerHTML = data;
}

data = { "name": "Sam Scott", "average": 99 };

fetch(
    "http://localhost/rest_api_basics_full/my/stuff?x=10",
    {
        method: "PUT",
        body: JSON.stringify(data)
    }
)
.then(response => response.text())
.then(success)
```

In order to run this code, you must have both folders (**rest_api_basics** and **test_rest_api_basics**) inside your **htdocs** folder. Then visit http://localhost/test_rest_api_basics/test.html.

Note that sending AJAX calls from a website on one domain to a server on a different domain can sometimes fail if the server does not allow **cross-origin** requests.

PHP Support

If you want to make sure you allow cross-origin requests (i.e., you want web sites on another domain to be able to access your web service via AJAX requests) then you should consider setting these access control headers in PHP:

```
header("Access-Control-Allow-Origin: *");
header("Access-Control-Allow-Methods: POST, GET, PUT, DELETE");
```

Python Support

Cloud computing resources are often accessed from desktop or mobile apps rather than from web apps. Most languages have straightforward support for sending the HTTP requests required by a web service.

In Python, you can use the **requests** module, as shown in the **test_rest_api_basics/test.py** file on Canvas. Here's an example of how to make a **PUT** request:

```
import json
import requests

data = { "name": "Sam Scott", "average": 99 }
url = "http://localhost/rest_api_basics_full/my/stuff?x=10"

r = requests.put(url, json.dumps(data))

print("Status Code:", r.status_code)
print(r.headers)
print(r.text)
```

To run this code, use your choice of Python 3 installation and IDE. You will need to have XAMPP running with **rest_api_basics** inside your **htdocs** folder, and you might need to install the Python **requests** module for the code to run. This can be done by typing the following command into the Python shell:

```
pip install requests
```

If the response text is JSON-encoded, you can use `json.loads` to convert it to an object.

```
data = json.loads(r.text)
```

Public APIs and Authentication

The Twitter API examples shown above cost money to access and require authentication, but there are lots of free APIs out there as well. Most of the free APIs are information services, so they only serve GET requests. Some of them are stricter than others in their interpretation of the REST standards.

There's a big list of free APIs here:

<https://github.com/public-apis/public-apis>

Some of the APIs in this list require authentication but many do not.

Example: Open-Meteo Weather

<https://open-meteo.com/> is a free-to-use API to retrieve weather information for a given latitude and longitude. If you're running JavaScript on a web page, you can use the JavaScript geolocation API to retrieve the user's coordinates and then send them to the open-meteo API to retrieve JSON-encoded weather data for them.

Go to w3schools (https://w3schools.com/html/html5_geolocation.asp) for info on how to use the geolocation API.

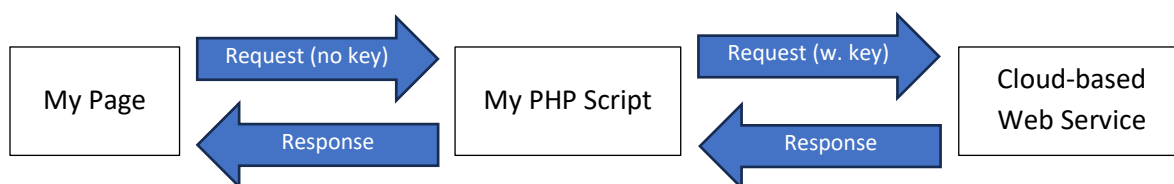
Example: NewsAPI.org

Most APIs (even free ones) will require you to register to use them, and then will require you to include authentication information whenever you send an HTTP Request. This is to give the API owners some control over who uses their service and helps prevent abuse. The most basic type of authentication is a private API key.

<https://newsapi.org/> is a service that allows GET requests to retrieve current news stories. You can sign up for free and get an API key. Then when you send a request, you must send that key in a query parameter called **apiKey**. Failure to send a valid API key will result in your request being denied.

Keeping API Keys Secret

If you have an API key for a web service, you must keep it secret otherwise someone could impersonate you, and possibly run up a big bill on your behalf. This means you should never send a web service request that includes an API key from JavaScript running on a web page because **view page source** will reveal your key! In this case, you need a three-tier architecture where your JavaScript program sends a generic request to a PHP program. Then the PHP program adds the API key and forwards it on to the web service. This architecture is beyond the scope of this course, but it's important to be aware of what would be required to hide an API key from prying eyes.



Exercises

1. Create a database table for “twits” (they’re like tweets but shorter – 40 characters maximum). Every twit should have an **ID**, a **UserID** (for the sender of the twit), and the **text** of the twit. Every **UserID** should be 8 digits long. Add a few rows to the table.

Now implement the following RESTful API, using **rest_api_basics** as a starting point:

Method	URL	Description
GET	/twits/all	Retrieve all tweets in the table. Should return a JSON-encoded array with the ID, UserID, and text of each tweet.
GET	/twits/:userid	Retrieve all tweets from the given :userid. Should return a JSON-encoded array with the ID and text of each tweet.

Test your API through the URL of a web browser.

2. Expand your twits API to add the following:

Method	URL	Description
POST	/twits/:userid	Adds a new tweet to the table for :userid. The text of the tweet is in the body of the request. Should return the ID of the new tweet.
DELETE	/twits/:id	Deletes the tweet by :id
PUT	/twits/:id	Changes the text of a tweet by :id. The new text is sent in the body of the request.

Test your API using Insomnia, making sure that the correct changes are made to the database table.

3. Expand the written specification of the API above to add two new API calls. These calls should allow a user to request the most recent twits and to search twits. It should be possible to use **query parameters** to specify how many of the most recent twits you want, and the text you wish to search for. Make sure you specify the correct Method and URL.
4. Implement the two new API calls you created for the last question.

5. Write a Python or JavaScript program that allows a user to view all the tweets they have added to the table (using their userid), and to add new tweets and see the result. The exact user interface for this is up to you.
6. Use Insomnia to experiment with the Open-Meteo API at <https://open-meteo.com/>. Then create a web page that displays at least the current temperature and windspeed based on the user's location, retrieved from the JavaScript geolocation API. You can base your code on the w3schools example:

https://www.w3schools.com/html/tryit.asp?filename=tryhtml5_geolocation
7. Register at <https://newsapi.org/> and obtain an API Key for the service. Experiment with the API using Insomnia. Then create a Python or JavaScript app that allows the user to retrieve and display headlines based on a search query.